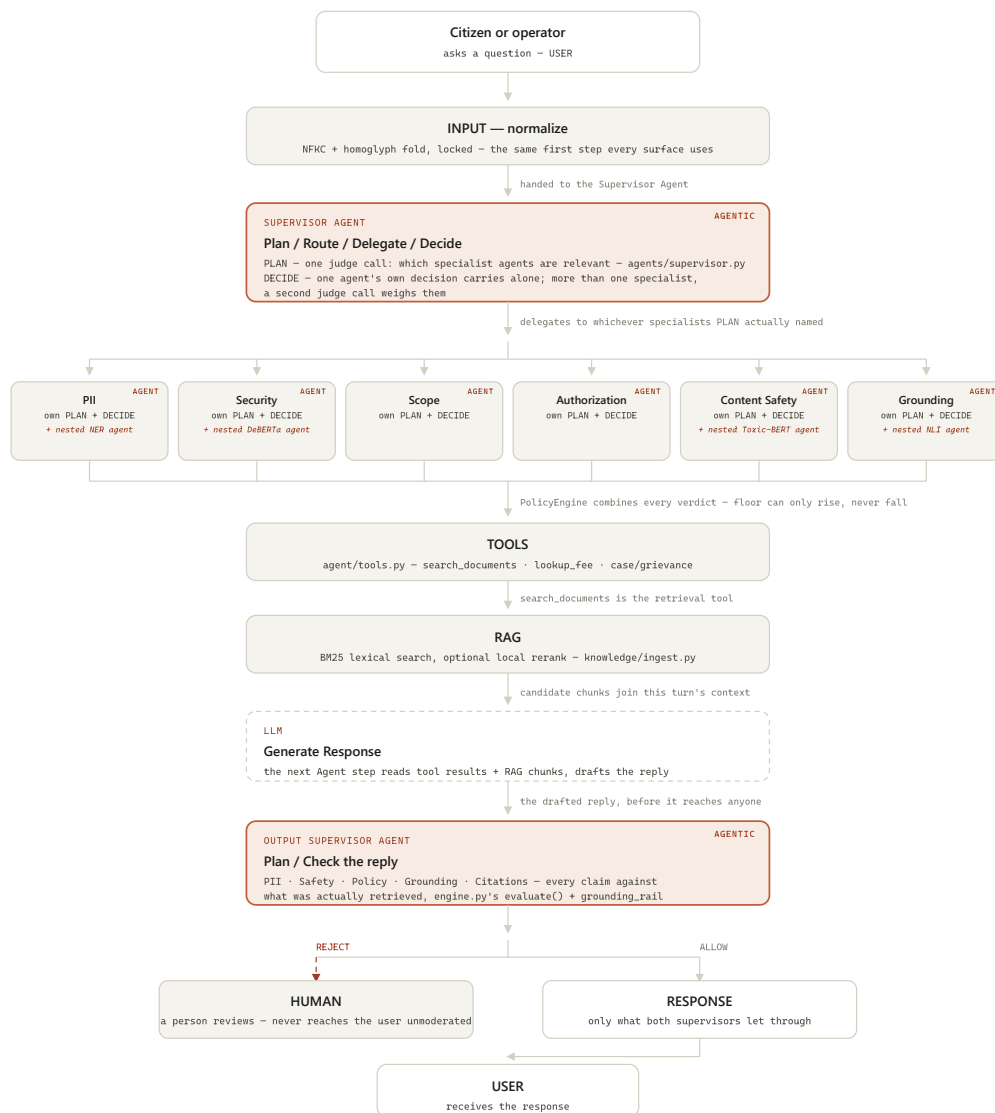


# The Guardrail Pipeline, At a Glance

One diagram, top to bottom: a question comes in, a Supervisor Agent decides which specialist checks it actually needs, the model answers, and a second Output Supervisor Agent checks the reply before it ever reaches the person who asked. This is the same diagram the product's own home page shows — not a simplified stand-in for it.



—— solid — deterministic code    --- dashed — a one-shot model call

— clay — a guardrail gate    --- red — where a request can stop short

**AGENT** its own PLAN + DECIDE, calling the judge itself

**AGENTIC** a judge plans this gate's own checklist

+ *nested ...* a local model, in its own PLAN+DECIDE agent

Two symmetric supervisor agents, not one gate at the front and a checklist at the back. The Supervisor Agent's own PLAN call decides which of the six specialists even run — an unrelated question never pays for a scope or authorization check it doesn't need. Each specialist is a full agent with its own nested PLAN/DECIDE loop before its own detector. The Output Supervisor Agent afterward is the same shape, applied to the reply instead of the request — including citations, which checks every claim against what `search_documents` actually returned. Only **REJECT** ever routes to a person; everything **ALLOW** reaches becomes the response. Four specialists — PII, Security, Content Safety, Grounding — go one level deeper still: their one local model (Presidio, DeBERTa, Toxic-BERT, an NLI model) is itself wrapped in a small nested agent with its own PLAN+DECIDE, rather than called as a flat, un-reasoned function. That nested agent's own decision carries no deterministic floor by default ( `agent.nested_model_floor` , off by default) — the parent specialist's own floor, unchanged, still governs the request either way.

## Where this runs today

One shared chain — `GuardrailSupervisor` → `Supervisor` → `PolicyEngine` — reachable from three doors in.

| ENTRY POINT                            | RUNS THE SUPERVISOR AGENT STAGE ABOVE?                                    | THEN WALKS                                                                                                                                                             |
|----------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST<br><code>/api/pipeline/run</code> | <code>always on</code>                                                    | Supervisor Agent → tools/RAG → LLM → Output Supervisor Agent — this diagram, in full. This is the live demo behind it.                                                 |
| POST <code>/api/chat</code>            | <code>on by default</code><br><code>supervisor.chat_prefilter_mode</code> | Supervisor Agent and the Output Supervisor Agent, then the ordinary retrieval → reply pipeline — ordinary chat has no tool loop, so Tools/RAG above don't apply there. |
| POST<br><code>/api/agent/chat</code>   | <code>on by default</code><br><code>agent.prefilter_mode</code>           | the full walk above, from inside the tool-using agent loop.                                                                                                            |

**Both prefilters default ON today.** Every ordinary chat turn and every agent turn already runs the Supervisor Agent stage before anything else — this isn't a demo-only path reserved for `/api/pipeline/run`, it's the current default behaviour of the whole app.

## Meet the agents

Nine agents in total: one that talks to the user and can act for them, six specialists that each judge one kind of risk, and two orchestrators that decide which specialists to run and reconcile what they find.

### Task Agent CHAT + TOOLS

Answers the citizen's question and can act for them — check a claim, file a grievance. Every write it does pauses for a person to approve first.

### Supervisor Agent FRONT GATE

Plans which of the six specialists below are worth running, delegates to them, then weighs what they found into one verdict.

### PII SPECIALIST

Finds personal identifiers — names, emails, SSNs, cards — and decides whether to mask, redact, or let them through. Its local model (Presidio) runs inside a nested agent with its own PLAN+DECIDE, not a flat lookup.

### Security SPECIALIST

Watches for prompt injection — attempts to override or extract the assistant's own instructions, direct or hidden in retrieved text. Its local model (DeBERTa) runs inside a nested agent the same way.

### Scope SPECIALIST

Checks whether the request is even something this assistant is meant to handle.

### Authorization SPECIALIST

Checks whether the request reaches for something that may belong to someone else, or asks to bypass access rules outright.

### Content Safety SPECIALIST

Screens for hate, violence, self-harm intent, and out-of-place content — while treating ordinary distress or frustration as normal traffic, not a signal. Its local model (Toxic-BERT) runs inside a nested agent the same way.

### Grounding SPECIALIST

Checks the drafted reply's own claims against what was actually retrieved — an unsupported claim is not delivered as fact. Its local model (an NLI entailment model) runs inside a nested agent the same way.

### Output Supervisor Agent BACK GATE

The same plan-then-check pass as the front gate, applied to the reply instead of the request: PII, safety, policy, grounding, and citations — before the user ever sees it.

---

GuardRails – one-page pipeline reference · diagram matches the live product's own home page, including the four nested local-model agents · generated from the code on disk, 2026-09-03 · see [guardrails-demo-script.html](#) for a full presenter walkthrough.